Choosing models. Explaining predictions.

AutoML, SHAP and explanation stability

How do we choose a hypothesis function?

Why does that function make this prediction?

First, what does one row represent?

A controlled synthetic customer example, not observed company records

One row is one customer. The target is whether that customer leaves.

The current computed example uses simulated, standardised inputs. Its values are not days, counts or months.

We must understand the inputs before interpreting a score or an explanation.

Give every input a meaning

Input	Intended meaning in the teaching example
Recency	How long since the customer last used the service
Inactivity	A duplicate of recency in this controlled simulation
Complaints	A simulated measure of customer complaints
Tenure	How long the customer has been a customer
Noise	An artificial random control, not a customer measurement

The selection rule comes before the search

Predict whether a customer will leave.

Eligible?
Mean CV recall ≥ 0.70

Among eligible candidates
Maximise mean CV precision

If no candidate meets the rule, report that. A search cannot invent an eligible solution.

We select a recipe. Training learns a function.

Feature option
Recency only

→

Tree settings
Depth 4, leaf 15

→

Fitted tree
Splits and predictions

Chosen by the search

Learned from training data

The candidate is a pipeline configuration. A fitted hypothesis function depends on training data too.

One candidate needs a fair evaluation



Fit within each fold, score on its validation portion, then average precision and recall separately.

Compare the candidate choices

All three candidates use decision trees; the selected inputs and settings differ.

Candidate	Activity inputs	Depth	Min. leaf
M0	Recency + inactivity	2	5
M43	Neither activity input	4	15
M19	Recency only	4	15

All three also include complaints, tenure and the artificial random control.

Compare their validation performance

Computed means over the same three validation folds

Candidate	Precision	Recall	Recall ≥ 70%?
M0	71.8%	87.2%	Yes
M43	68.4%	56.4%	No
M19	83.2%	78.6%	Yes

First check eligibility. Then compare precision among the eligible candidates.

Select the best eligible candidate shown

M19: recency, depth 4, minimum leaf size 15.

It has the highest mean validation precision among the eligible candidates in this comparison.

This is a selection result for these candidates, not proof of the best possible model.

Choosing, fixing and repeating are different

Choose a configuration

Fix that configuration

Repeat training runs

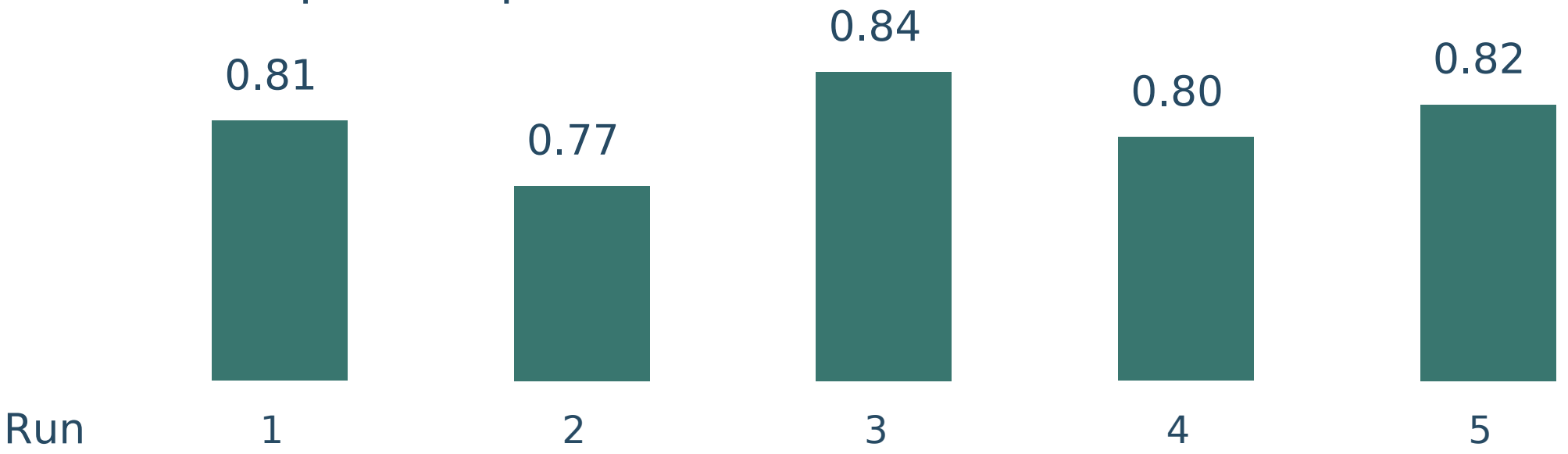
Repeating training measures variation for a fixed recipe. Repeating the entire search also allows the chosen recipe to change.

State what changes: training seed, data split, search seed, or the entire procedure.

Last time: one run was one outcome

Illustrative repeated runs of the same experimental procedure

The seed repeats a procedure. Several runs reveal variation.



Keep the configuration fixed and state exactly what changes between runs.

The search space is a set of possible recipes

4 feature options × 4 depths × 3 leaf sizes = 48 pipelines

	Depth 2		Depth 3		Depth 4		Depth 5					
Recency + inactivity	0	1	2	3	4	5	6	7	8	9	10	11
Recency only	12	13	14	15	16	17	18	19	20	21	22	23
Inactivity only	24	25	26	27	28	29	30	31	32	33	34	35
Exclude both	36	37	38	39	40	41	42	43	44	45	46	47

Each depth has leaf sizes 5, 15 and 30. Each highlighted square is one complete configuration.

AutoML automates parts of this selection process

We specify

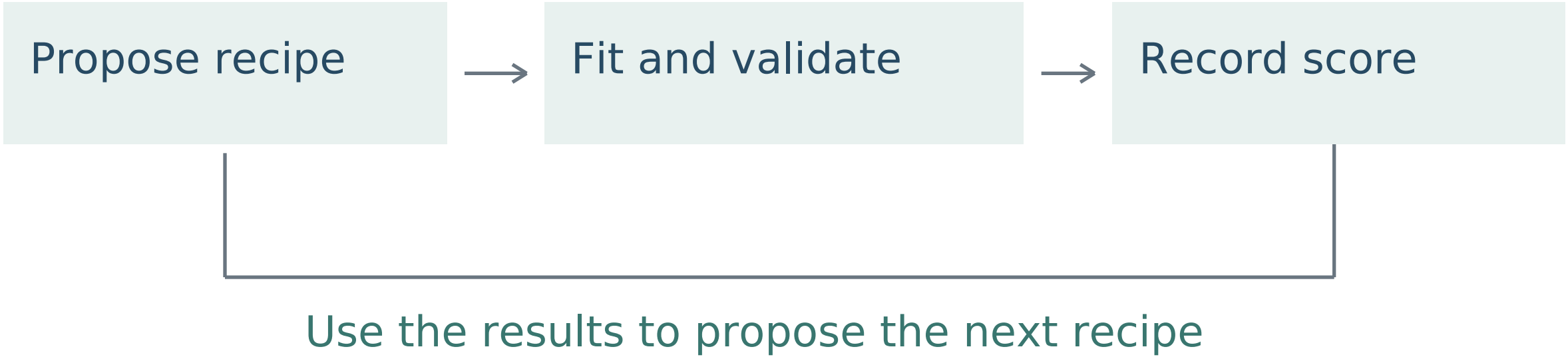
The task, available inputs, validation design, objective and budget.

The system searches

It proposes recipes, fits candidates and uses scores to decide what to try next.

AutoML can include preprocessing, feature choices, model families and hyperparameters.

The search loop surrounds model fitting



Stop at the declared budget. Keep the best eligible candidate found.

Frameworks package the machinery

Examples of different design choices

Framework	Search idea	What it helps organise
auto-sklearn	Bayesian optimisation	Pipeline search and ensembles
TPOT[1]	Evolutionary search	Pipeline construction and evaluation

[1] TPOT: Tree-based Pipeline Optimization Tool. Defaults still express modelling choices.

Automation inherits our decisions and mistakes

If we supply...	The consequence can be...
A poor metric	A strong score for the wrong goal
An incomplete search space	No access to a useful candidate
Leaked or unsuitable validation	Efficient selection of misleading results
A small budget	A result shaped by the visited candidates

Simulated annealing explores one candidate at a time

Current recipe A



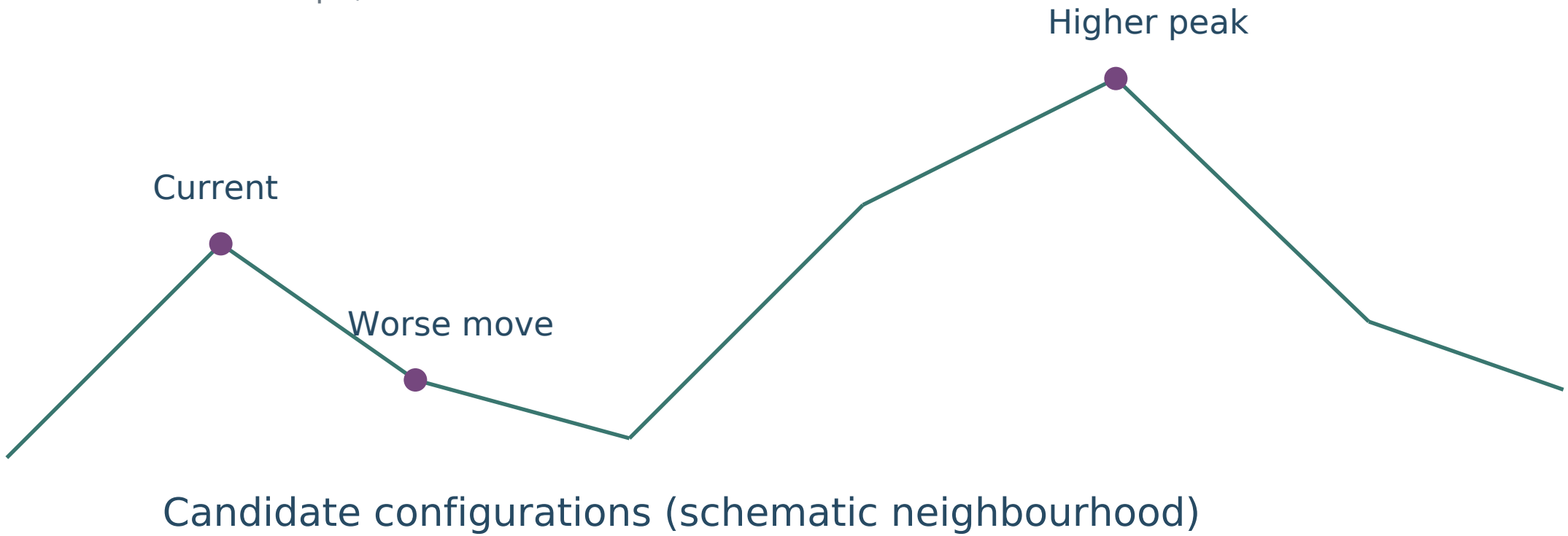
Proposed neighbour B

Change one choice, such as tree depth 3 to 4. Train and validate the new recipe using the same folds.

Keep the best eligible candidate seen, even when the current candidate moves elsewhere.

Sometimes a worse move helps escape a local peak

Schematic score landscape, not measured model results



A greedy search rejects every worse move. Annealing can cross a lower-scoring region.

Cooling makes worse moves less likely

For two eligible candidates, let the score be CV precision.

Precision decreases by 0.02	Temperature T	Accept with probability
Early exploration	0.05	exp(-0.02 / 0.05) ≈ 67%
Later exploration	0.005	exp(-0.02 / 0.005) ≈ 2%

Better moves are accepted. Define feasibility handling, cooling schedule and budget in advance.

Why consider evolutionary search?

The choices are mixed

Feature subsets, model families and integer settings do not form one smooth surface.

A population explores alternatives

Selection uses validation feedback. Variation creates new recipes.

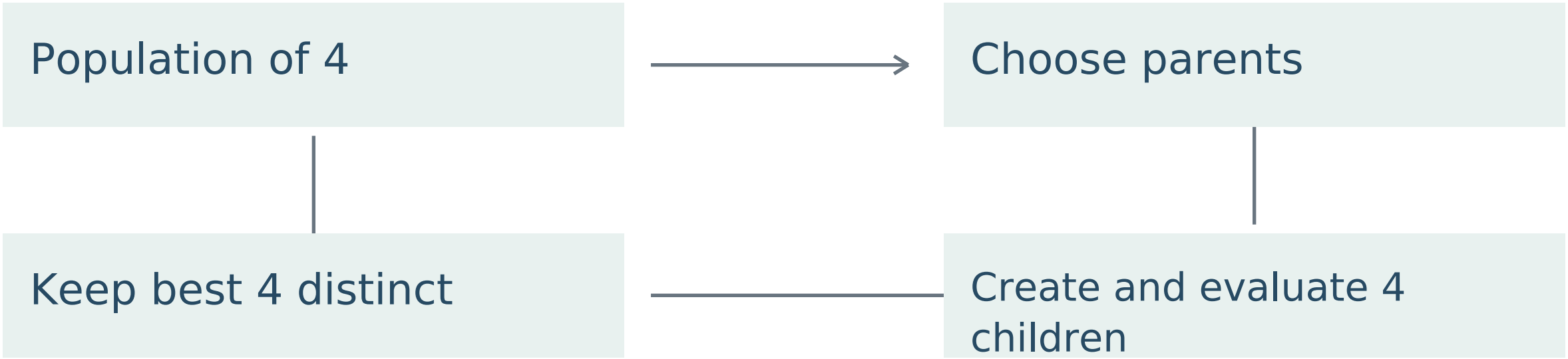
Evolution still costs evaluations, can converge early and needs comparison with simple baselines.

A candidate can be encoded as a chromosome

Feature choice	Maximum depth	Minimum leaf size
Recency only	4	15

Changing a gene changes the recipe. We must train and evaluate the new candidate.

Selection, variation, evaluation, survival



A population is a set of candidate pipelines. A generation is one round of this loop.

Recombination and mutation change the recipe

Candidate	Features	Depth	Leaf
Parent A	Recency only	4	15
Parent B	Recency + inactivity	2	30
Recombined child	Recency only	2	30
After mutation	Recency only	5	30

A search result depends on its budget and seed

Hold fixed

Candidate space, folds, scoring rule and evaluation budget.

Repeat the search

Change search seeds. Record outcomes, failures and evaluation costs.

This 48-candidate demo explains the mechanism. It cannot establish that evolution is superior.

A good score leaves another question

Why did this fitted model make this prediction?

Customer 1

Selected model → Leave: 90.8%

An explanation describes how the model uses information to produce an output.

Explain one prediction first

A small illustrative example with two customer features

Customer information

Complaints: 3 this month
Tenure: 2 months as a customer

Predicted chance of leaving

70%

These values illustrate the calculation. They are separate from the 48-pipeline experiment.

Start with the model’s usual prediction

Choose a group of reference customers. Run the same fitted model on each one and average its predictions.

Reference average
30%

This customer
70%

The baseline is an average model prediction, not necessarily the percentage who actually leave.

The explanation accounts for the 40-point gap

70% − 30% = 40 percentage points

Complaints
+25 points

Tenure
+15 points

30% baseline + 25 points + 15 points = 70% prediction.

Read the explanation as a running total

Baseline
30%

→

Add complaints
55%

+25 points

→

Add tenure
70%

+15 points

The bars explain this model’s prediction. They do not show what would happen if we changed the customer.

Where do the contributions come from?

Keep the fitted model fixed. Compare predictions with different pieces of this customer’s information supplied.

For the other feature, use values from the reference customers and average the predictions.

We do not retrain the model or delete an input column.

Four predictions are enough for this example

Illustrative averages from the same fitted model and reference group

Customer information supplied	Average prediction
Neither feature	30%
Complaints = 3 only	50%
Tenure = 2 months only	40%
Complaints = 3 and tenure = 2 months	70%

A feature’s contribution depends on the order

Complaints first

30% → 50%
Complaints adds 20 points

Complaints after tenure

40% → 70%
Complaints adds 30 points

SHAP averages these two contributions: (20 + 30) / 2 = 25 points for complaints.

Do the same for tenure

Tenure first

30% → 40%
Tenure adds 10 points

Tenure after complaints

50% → 70%
Tenure adds 20 points

Average: (10 + 20) / 2 = 15 points for tenure. Together the contributions explain the 40-point gap.

This is the idea behind SHAP

SHapley Additive exPlanations

For each feature, average how much it changes the prediction when added in different orders.

Baseline + all feature contributions = prediction

With more features there are more orders. Algorithms calculate or estimate the contributions.

A contribution can also lower the prediction

Positive contribution
Raises the output

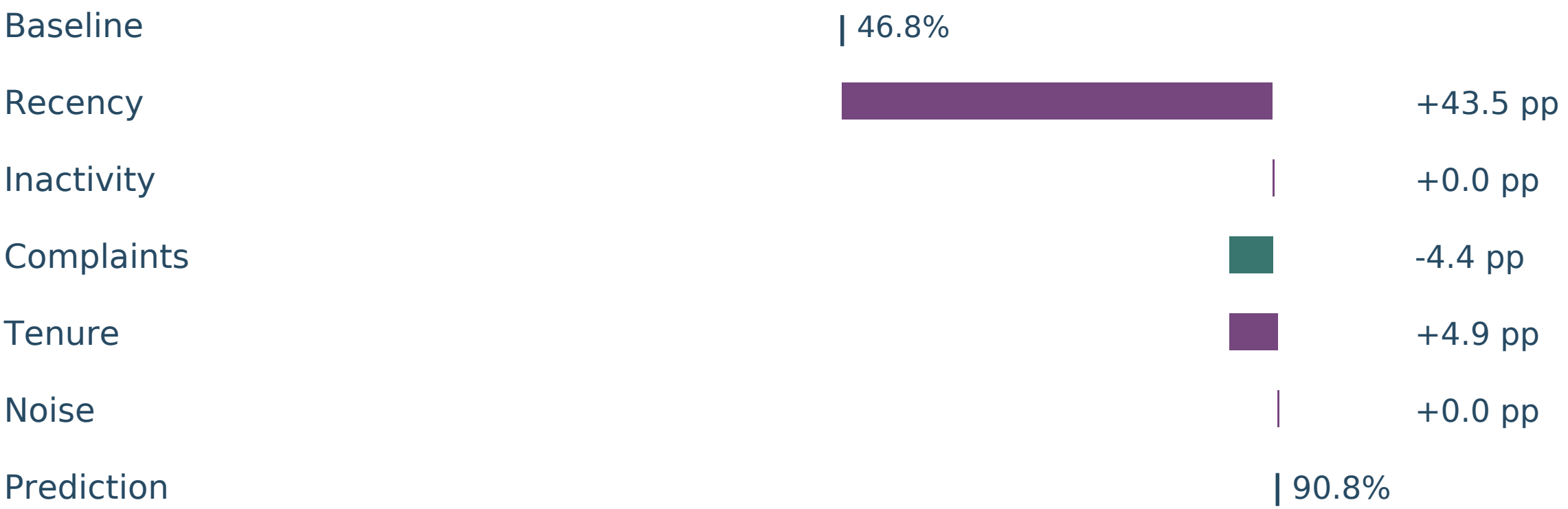
Negative contribution
Lowers the output

The sign describes the contribution to this prediction relative to the baseline.

A negative SHAP value does not mean that the feature itself has a negative value.

Now read the explanation for the computed model

M19 and customer 1 from the synthetic example



One customer and many customers answer different questions

One customer

Which features raise or lower this prediction?

Many customers

Which features receive large contributions most often?

A common summary averages the absolute contribution for each feature across customers.

What an explanation does and does not establish

It describes how a particular model uses the inputs for a prediction.

It depends on the reference customers and how missing information is represented.

It does not establish that a feature causes customers to leave.

What we can now explain

The search selects a pipeline configuration using validation evidence.

SHAP allocates a fixed model output relative to a declared reference.

Now a research question: would another similarly performing model tell the same story?

Several hypothesis functions can perform well

	Depth 2		Depth 3			Depth 4			Depth 5			
Recency + inactivity	0	1	2	3	4	5	**6**	**7**	8	9	**10**	11
Recency only	12	13	14	15	16	17	**18**	**19**	20	21	**22**	23
Inactivity only	24	25	26	27	28	29	**30**	**31**	32	33	**34**	35
Exclude both	36	37	38	39	40	41	42	43	44	45	46	47

Several candidate pipelines can fit nearly as well under the same evaluation rule.

The Rashomon idea keeps near-optimal alternatives

Keep eligible candidates within 2 precision points of the best tested eligible candidate.

Best observed	Cutoff	In the set
83.29%	81.29%	9 of 48

Similar scores can support different attributions

M19: Recency only

Precision 83.25%
Prediction 90.8%

Recency: +43.5 points

Inactivity: +0.0 points

M31: Inactivity only

Precision 83.25%
Prediction 90.8%

Recency: +0.0 points

Inactivity: +43.5 points

Same customer, background and SHAP definition. Only the fitted model changes.

Stability requires saying what varies

Change	Keep fixed to isolate it
Training sample or seed	Recipe, audit cases, explanation settings
Selected model	Audit cases, background, SHAP definition
Background sample	Fitted model and cases

Persistent attribution is evidence of robustness to that change. It does not establish causal truth.

The whole experiment needs an honest claim

We searched a declared space under a fixed budget.

We explained a specified output with a specified reference.

We checked whether the explanation persisted across alternatives.

Similar predictive performance does not guarantee similar feature contributions.

Sources for AutoML and explanation

Olson and Moore (2016). TPOT: A Tree-based Pipeline Optimization Tool.

https://proceedings.mlr.press/v64/olson_tpot_2016.html

Lundberg and Lee (2017). A Unified Approach to Interpreting Model Predictions.

https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

Fisher, Rudin and Dominici (2019). All Models are Wrong, but Many are Useful.

https://jmlr.org/papers/v20/18-760.html

Example methods and further reading

Synthetic independent customers. Shared 3-fold CV on 900 development cases. Fixed tree seed. A separate audit set supports exploratory explanations. The reserved test set stays unused.

Exact Shapley values enumerate 32 feature subsets using 60 fixed background rows. Local additivity is checked for every model and audit case.

Framework documentation: automl.github.io/auto-sklearn/master/manual.html
TPOT: epistasislab.github.io/tpot/latest/
SHAP: shap.readthedocs.io/en/latest/generated/shap.TreeExplainer.html